---

1. `yolov8_weed_detector.py` (Vision & AI)

What is Faked/Placeholded:

- **Fake Detections:** The function `_simulate_detections()` uses `np.random` to generate fake bounding boxes, confidence scores, and classes if a YOLO `.pt` file is not found.
- **Fake Depth Map:** In the main block, it generates a fake depth image using `np.random.uniform(0.3, 1.5)`.

What is Missing (Claimed in paper but absent in code):

- **Visual Output:** There is no code to draw the bounding boxes on the image and save them (which is required for **Figure 5**).
- **Metrics Calculation:** The paper claims specific mAP scores, Precision-Recall curves, and a Confusion Matrix (**Figure 4**). There is no code here to evaluate the model against a test dataset to generate these graphs.

What needs to be done:

1. Download a public dataset (e.g., *DeepWeeds* or *CottonWeedID15*).

2. Write a separate training script to train YOLOv8-nano and get a real `weeds.pt` file.

3. Add `cv2` (OpenCV) code to draw the bounding boxes, overlay the 3D depth coordinates as text on the image, and save the output.

4. Write an evaluation function using the `ultralytics` library to output the Confusion Matrix and Precision-Recall curves.

---

2. `delta_robot_kinematics.py` (Robotic Arm)

What is Faked/Placeholded:

- **Basic Testing:** The `__main__` block just tests 5 hardcoded (X, Y, Z) coordinates and prints text to the terminal.

What is Missing (Claimed in paper but absent in code):

- **PID Controller:** The paper claims PID tuning (Kp=2.4, Ki=0.18, Kd=0.07) and shows a step-response graph (**Figure 3b**). There is absolutely zero PID control code in this file.
- **Nearest-Neighbor Optimization:** The paper claims a "greedy nearest-neighbor heuristic" to find the shortest path between multiple weeds. The `plan_weed_removal_trajectory` function just processes weeds in the exact order they are given.
- **Plotting:** No code to plot the 3D workspace cross-section (**Figure 2a**), joint angle profiles over time (**Figure 2b**), or the XY tool path (**Figure 3a**).

What needs to be done:

1. Write a script that samples thousands of X, Y, Z points, runs Inverse Kinematics, and plots the reachable points in a 3D `matplotlib` graph (Workspace plot).

2. Write a simple simulated PID function to generate the step-response curve graph.

3. Add the Nearest-Neighbor math (using `scipy.spatial.distance` or simple Pythagoras) to sort the `weed_positions` array before calculating the trajectory.

4. Calculate the kinematics over time (t=0 to 2.5s) and plot the $\theta_1, \theta_2, \theta_3$ angles as line graphs.

---

3. `mobile_robot_navigation.py` (Navigation & Mapping)

What is Faked/Placeholded:

- **Fake Sensors:** Inside `__main__`, the GPS is faked using `np.random.normal(0, 0.02)`. The IMU update is fed hardcoded zeros: `ekf.update_imu(0.0, 0.0, 0.0)`.

What is Missing (Claimed in paper but absent in code):

- **Obstacle Avoidance:** The paper claims the robot uses "VFH+ (Vector Field Histogram)" and LiDAR. There is no LiDAR processing or obstacle avoidance code here whatsoever.
- **Crab-Walking (Holonomic Motion):** The paper claims the omnidirectional wheels allow it to switch rows without rotating. However, the Boustrophedon planner code literally turns the robot 180 degrees (`theta=math.pi`).
- **Plotting:** No code to draw the field, the serpentine path, or the EKF error graph (**Figure 6a and 6b**).

What needs to be done:

1. Change the Boustrophedon planner so that `theta` always remains `0.0` to reflect actual omnidirectional crab-walking.

2. Write a "Sensor Simulator" function that takes the perfect planned path, adds mathematical noise to simulate real GPS/IMU, feeds it to the EKF, and logs the difference (error) over time.

3. Write `matplotlib` code to plot the 2D field, the planned path, and the EKF error graph.

4. *Optional:* Either remove the LiDAR/VFH+ claims from your paper entirely, or write a dummy obstacle avoidance function to justify leaving it in the text.

---

4. `main_robot_controller.py` (The State Machine)

What is Faked/Placeholded:

- **Fake Environment Feed:** The main loop feeds completely random matrices (`np.random.randint`) into the vision system every 5 steps.
- **Fake Movement Execution:** It "steps" navigation instantly without simulating velocity, acceleration, or time realistically.
- **Fake Battery:** It just blindly subtracts `0.5` from the battery every step.

What is Missing (Claimed in paper but absent in code):

- **Energy and Power Modeling:** The paper shows a detailed battery discharge curve and power breakdown by subsystem (motors vs. computer vs. arm) in **Figure 8**. The code does not track power consumption in watts.
- **Latency Breakdown:** The paper claims a 54.1 ms total latency (Detection + IK + Control). The code measures total runtime but doesn't log the microsecond latency of each individual step to generate **Figure 3c**.
- **Data Export:** The `_session_log` records text but never exports it to a `.csv` for analysis.

What needs to be done:

1. Modify the loop to process real images from a folder (your public dataset) instead of random pixel arrays.

2. Implement a `PowerModel` class that calculates Watt-hours consumed based on state (e.g., IDLE = 50W, REMOVING = 233W) to generate realistic battery drain data.

3. Add Python `time.perf_counter()` around the Detection, IK, and Path Planning functions, log these times, and export them to a CSV so you can plot the Latency Breakdown bar chart.

4. Export the final simulated trial stats to a Pandas DataFrame/CSV so you can fill out **Table 4** with your new simulation metrics.